

7. Protection and Security

1. Protection and Security
2. Protection mechanism (Access control list, capability list)
3. User authentication,
4. Frauds and attacks,
5. Trusted system

1. Protection and Security

- **Protection:**
 - Focuses on ensuring proper resource usage within a computer system.
 - Prevents unauthorized access or accidental misuse of resources (files, memory, CPU).
 - Operates through mechanisms such as **permissions, privileges, and access control**.
 - **Security:**
 - Broader than protection, as it addresses external threats like cyberattacks, malware, and data breaches.
 - Deals with ensuring **confidentiality, integrity, and availability** of system resources.
 - Includes mechanisms such as **firewalls, encryption, and intrusion detection systems**.
-

2. Protection Mechanisms

- **Access Control List (ACL):**
 - An ACL is a list associated with a resource (e.g., a file or directory).
 - Each entry specifies a user or group and the permissions they have (e.g., read, write, execute).
 - Example:

File: report.txt
User1: Read, Write
User2: Read
 - Pros:
 - Fine-grained control over who can access resources.
 - Easy to manage for specific objects.
 - Cons:
 - Can become cumbersome if there are many users or objects.

- **Capability List:**

- A capability is a token or key held by users or processes, indicating the actions they can perform on objects.
- Stored as part of the user's session or process descriptor.
- Example:

```
User1: {File1: Read, Write; File2: Read}           // object{ }
User2: {File1: Read}
```

- Pros:
 - Faster lookup compared to ACLs in large systems.
 - Delegation of access is easier (users can pass capabilities to others).
 - Cons:
 - Requires secure mechanisms to ensure capabilities aren't forged or stolen.
-

3. User Authentication

- **Purpose:**

- Ensures that the entity attempting to access the system is legitimate.

- **Common Methods:**

- **Passwords:**

- ▢ Simple and widely used, but prone to attacks (e.g., brute force, dictionary attacks).
- ▢ Enhanced by using salted hashes for storage.

- **Biometric Authentication:**

- ▢ Uses unique physical traits like fingerprints, facial recognition, or iris scans.
- ▢ Advantages: Hard to replicate.
- ▢ Disadvantages: High cost and potential privacy concerns.

- **Two-Factor Authentication (2FA):**

- ▢ Combines two factors:
 1. Something the user knows (password).
 2. Something the user has (smartphone, security token).
- ▢ Increases security by requiring multiple independent proofs of identity.

- **Advanced Authentication:**

- Multi-Factor Authentication (MFA): Includes additional factors like "something the user is" (biometrics) or "something the user does" (behavioral patterns).
-

4. Frauds and Attacks

- **Frauds:**

- Misusing systems for unauthorized gain.
- Examples:
 - ▢ Financial fraud (e.g., credit card theft).
 - ▢ Identity theft.

- **Attacks:**
 - **Phishing:**
 - ▢ Fake websites or emails tricking users into sharing sensitive information.
 - ▢ Prevention: User education, spam filters.
 - **Malware:**
 - ▢ Malicious software like viruses, worms, and ransomware.
 - ▢ Prevention: Antivirus software, regular updates.
 - **Denial-of-Service (DoS):**
 - ▢ Flooding a system with traffic to make it unavailable.
 - ▢ Distributed DoS (DDoS): Involves multiple machines (e.g., botnets).
 - ▢ Prevention: Load balancers, rate limiting.
 - **Man-in-the-Middle (MITM):**
 - ▢ Intercepting communication between two parties.
 - ▢ Prevention: Using encrypted connections (e.g., HTTPS).
-

5. Trusted System

- **Definition:**
 - A system that reliably enforces a security policy.
 - Ensures confidentiality, integrity, and availability (the CIA triad).
 - **Features:**
 - **Confidentiality:** Prevents unauthorized disclosure of information.
 - **Integrity:** Protects data from unauthorized modification.
 - **Availability:** Ensures system resources are accessible when required.
 - **Key Concepts:**
 - **Reference Monitor:**
 - ▢ Mediates all access requests between subjects (users/processes) and objects (files/resources).
 - ▢ Must be tamper-proof, always invoked, and verifiable.
 - **Security Models:**
 - ▢ **Bell-LaPadula Model:** Focuses on maintaining confidentiality (no read-up, no write-down).
 - ▢ **Biba Model:** Focuses on data integrity (no read-down, no write-up).
 - **Evaluation Criteria:**
 - ▢ Trusted systems are often evaluated using standards like **Common Criteria** or **TCSEC (Orange Book)**.
-

Summarized

1. Protection and Security

Protection and security are fundamental concepts in operating systems, aiming to safeguard resources and data from unauthorized access, modification, or destruction. They involve a combination of hardware and software mechanisms to implement robust security measures.

Goal	Threat
Data confidentiality	Exposure of data
Data integrity	Tampering with data
System availability	Denial of service
Exclusion of outsiders	System takeover by viruses

Figure 9-1. Security goals and threats.

2. Protection Mechanisms

- **Access Control Lists (ACLs):** ACLs define permissions for each object, specifying which users or groups can perform specific actions (e.g., read, write, execute).

Protection Domains (1)

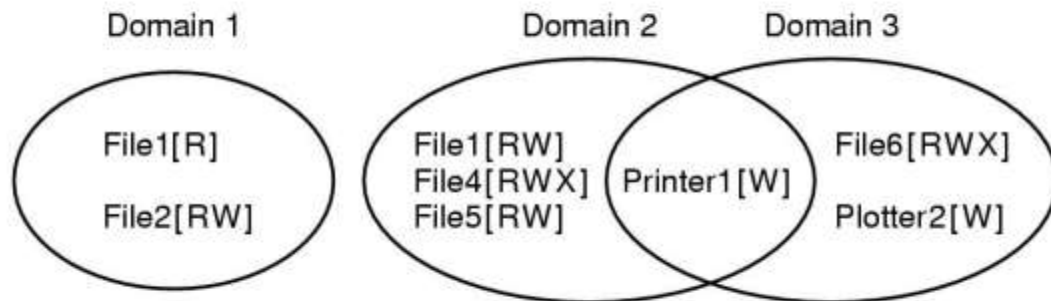


Figure 9-4. Three protection domains.

Protection Domains (2)

Domain	Object							
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2
1	Read	Read Write						
2			Read	Read Write Execute	Read Write		Write	
3						Read Write Execute	Write	Write

Figure 9-5. A protection matrix.

Protection Domains (3)

Domain	Object									
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2	Domain1	Domain2
1	Read	Read Write								Enter
2			Read	Read Write Execute	Read Write		Write			
3						Read Write Execute	Write	Write		

Figure 9-6. A protection matrix with domains as objects.

Access Control Lists (1)

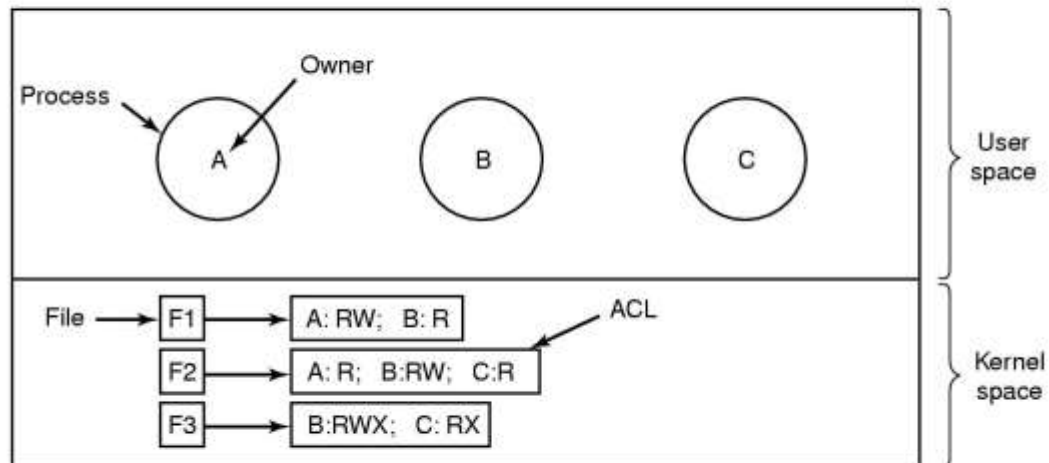


Figure 9-7. Use of access control lists to manage file access.

Access Control Lists (2)

File	Access control list
Password	tana, sysadm: RW
Pigeon_data	bill, pigfan: RW; tana, pigfan: RW; ...

Figure 9-8. Two access control lists.

- **Capability Lists:** Capability lists associate objects with their access rights, granting subjects permissions to access those objects.

Capabilities (1)

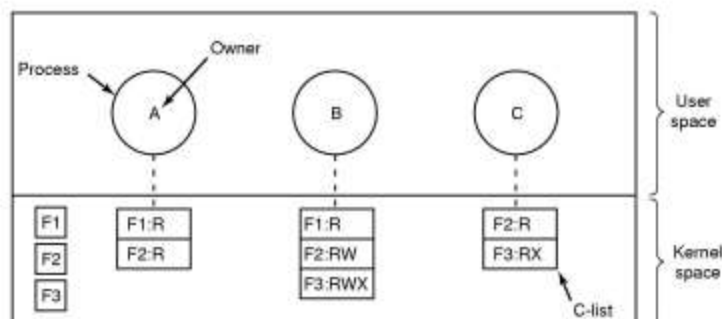


Figure 9-9. When capabilities are used, each process has a capability list.

Capabilities (2)

Server	Object	Rights	$f(\text{Objects}, \text{Rights}, \text{Check})$
--------	--------	--------	--

Figure 9-10. A cryptographically protected capability.

Capabilities (3)

Examples of generic rights:

- Copy capability: create a new capability for the same object.
- Copy object: create a duplicate object with a new capability.
- Remove capability: delete an entry from the C-list; object unaffected.
- Destroy object: permanently remove an object and a capability.

3. User Authentication

User authentication is the process of verifying a user's identity before granting access to system resources. Common methods include:

- **Password-based authentication:** Users provide a username and password.
- **Biometric authentication:** Physical characteristics like fingerprints or facial recognition are used.
- **Token-based authentication:** Users possess a physical or virtual token to prove their identity.

Authentication

General principles of authenticating users:

- Something the user knows.
- Something the user has.
- Something the user is.

Authentication Using Passwords

LOGIN: mitch
PASSWORD: FooBar!-7
SUCCESSFUL LOGIN

(a)

LOGIN: carol
INVALID LOGIN NAME
LOGIN:

(b)

LOGIN: carol
PASSWORD: Idunno
INVALID LOGIN
LOGIN:

(c)

Figure 9-17. (a) A successful login.
(b) Login rejected after name is entered.
(c) Login rejected after name and password are typed.

Authentication Using a Physical Object

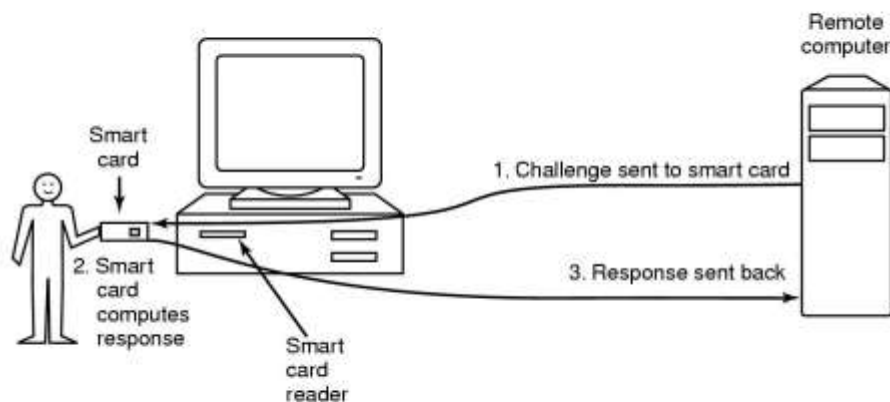


Figure 9-20. Use of a smart card for authentication.

Login Spoofing



Figure 9-23. (a) Correct login screen. (b) Phony login screen.

Authentication Using Biometrics

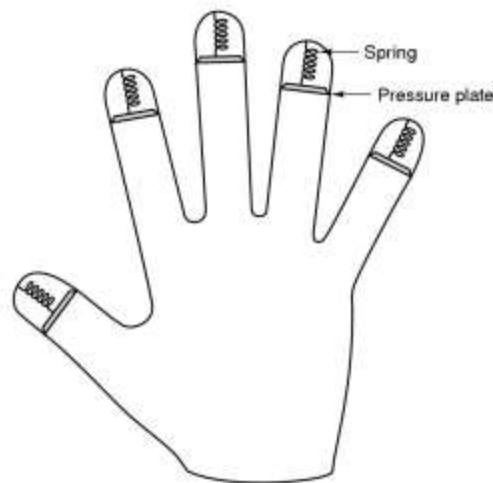


Figure 9-21. A device for measuring finger length.

4. Frauds and Attacks

Operating systems are susceptible to various frauds and attacks, including:

- **Malware:** Malicious software designed to harm systems.
- **Phishing:** Deceiving users into revealing sensitive information.
- **Denial-of-Service (DoS) attacks:** Overwhelming a system with traffic to prevent legitimate access.
- **Man-in-the-middle attacks:** Intercepting communication between two parties.

5. Trusted Systems

A trusted system is one that can be relied upon to perform its intended functions correctly and securely. Key characteristics include:

- **Security policy:** A formal statement of the security goals and constraints.
- **Security model:** A mathematical or abstract representation of the security policy.
- **Security mechanisms:** The implementation of security measures to enforce the policy.

Trusted Systems

- Consider reports of viruses, worms, etc.
- Two naive (but logical) questions:
 - Is it possible to build a secure computer system?
 - If so, why is it not done?

Trusted Computing Base

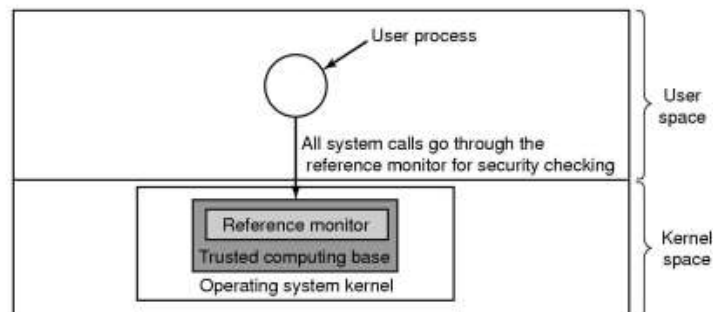


Figure 9-11. A reference monitor.

Key Considerations

- **Layered security:** Employing multiple layers of defense to protect against various threats.
- **Regular updates:** Keeping the operating system and software up-to-date with security patches.
- **User education:** Training users to recognize and avoid security risks.
- **Incident response plan:** Having a plan in place to respond to security breaches.

<https://slideplayer.com/slide/5174287/>
<https://slideplayer.com/slide/11818864/>

<https://slideplayer.com/slide/5174287/>
<https://users.soe.ucsc.edu/~sbrandt/111/Slides/chapter9.pdf>

Input/Output Chapter 5

<https://slideplayer.com/slide/11539058/>

<https://pages.cs.wisc.edu/~solomon/cs537-old/last/security.html>

<https://inf107.telecom-paris.fr/lectures/lecture-part3-4-fs.pdf>

<https://www.cs.fsu.edu/~zwang/files/cop4610/Fall2016/chapter12.pdf>

<https://www.slideshare.net/WajeehaBaig/file-system-237607939>

https://github.com/chlebig/ModernOperatingSystems_AndrewTanenbaum
https://mrcet.com/downloads/digital_notes/IT/R17A0513%20OS.pdf